

Bi-LSTM 기반 한국어 영화 리뷰 감성 분석

3조 - 김세영 최도경 최홍준 남주혜

CONTENTS

01 Bi-LSTM 개념 설명

02 Bi-LSTM 개념의 적용

03 데이터 전처리 및 모델 학습

04 예측 테스트를 통한 결과 확인

05 Bi-LSTM + Attention 모델

06 결론

01 Bi-LSTM 란?

일반적인 LSTM

왼쪽에서 오른쪽으로 정보 추출

eg.) 그 → 영화 → 솔직히 → ... → 재밌었다

결과물이 직전 패턴을 기반으로 수렴한다는 경향을 보임.



Bi-LSTM

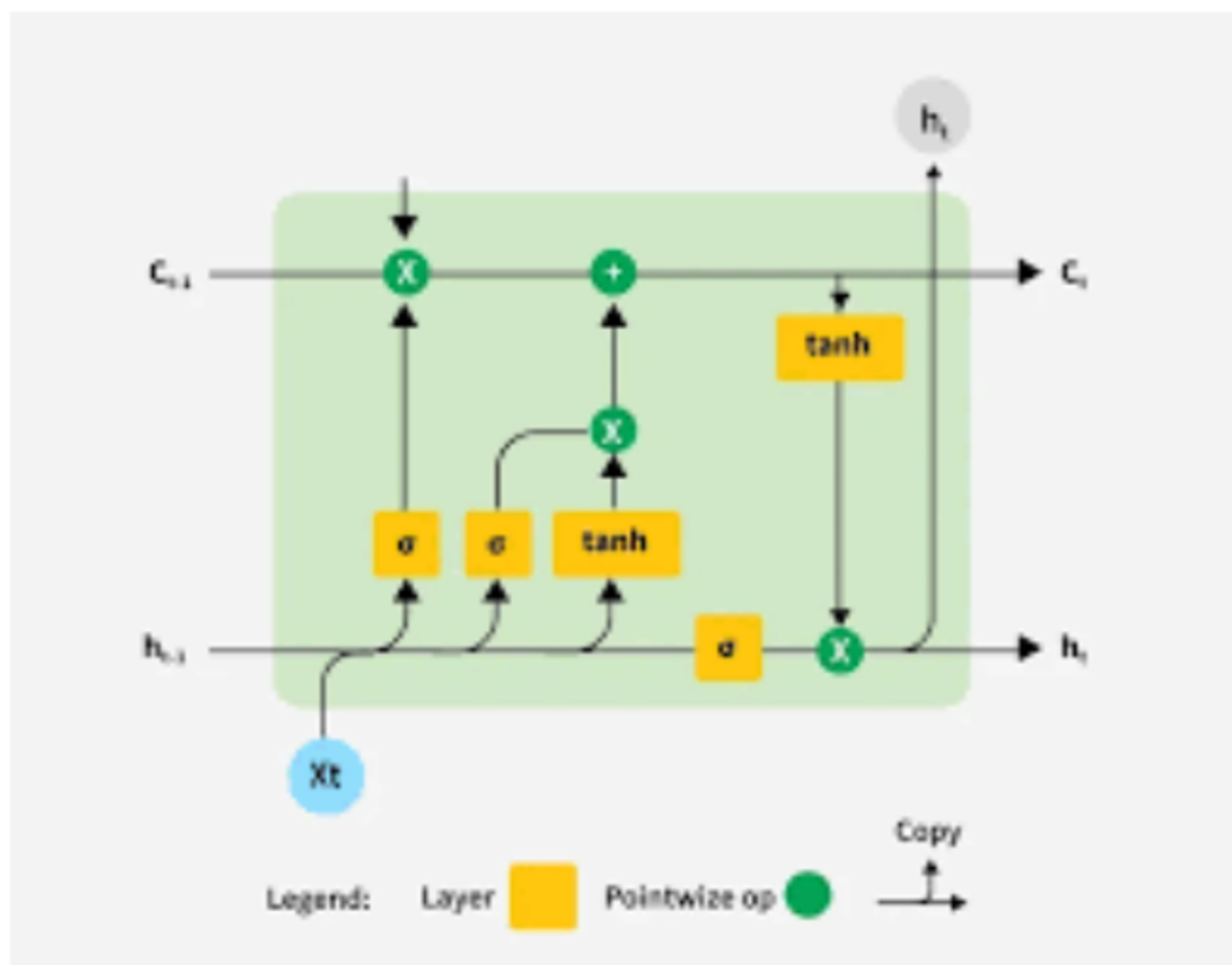
양방향으로 정보 추출 가능

eg.) 재밌었다 → 정말 → 했는데 → ... → 영화 → 그

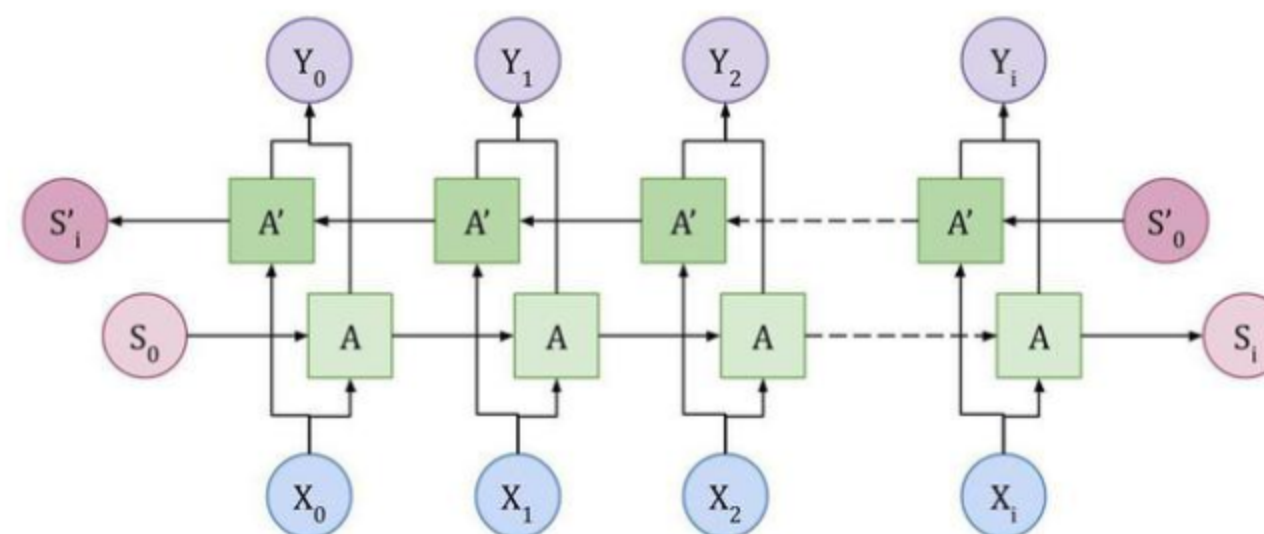
시퀀스 데이터에서 더 많은 정보를 추출할 수 있기 때문에
보다 높은 성능을 갖춤

01 Bi-LSTM 란?

일반적인 LSTM



Bi-LSTM



02 Bi-LSTM 개념의 적용

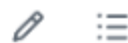
감성 분석은 문장의 단순한 단어 빈도가 아닌 문맥과 의미의 반전을 파악해야하는 복잡한 작업이므로 단방향 LSTM보단 Bi-LSTM이 유리함

연기는 좋았지만, 스토리가 별로였다 ⇒ 부정

- 단방향 LSTM : '좋았지만'까지 긍정의 기억을 쌓다가 '별로였다'를 통해 갑작스럽게 기억을 뒤집어야 함
- Bi-LSTM : 정방향 뿐만 아니라 역방향도 존재해 결론을 먼저 읽고 판단에 반영해 의미 반전 감지

NSMC : 네이버 영화 리뷰 데이터

📖 README



Naver sentiment movie corpus v1.0

This is a movie review dataset in the Korean language. Reviews were scraped from [Naver Movies](#).

The dataset construction is based on the method noted in [Large movie review dataset](#) from Maas et al., 2011.

Data description

- Each file is consisted of three columns: `id`, `document`, `label`
 - `id` : The review id, provided by Naver
 - `document` : The actual review
 - `label` : The sentiment class of the review. (0: negative, 1: positive)
 - Columns are delimited with tabs (i.e., `.tsv` format; but the file extension is `.txt` for easy access for novices)
- 200K reviews in total
 - `ratings.txt` : All 200K reviews
 - `ratings_test.txt` : 50K reviews held out for testing
 - `ratings_train.txt` : 150K reviews for training

3개의 컬럼 id, document, label로 이루어짐

id : Naver에서 제공한 리뷰 아이디

document : 실제 리뷰

label : 긍정, 부정에 대한 라벨

03 데이터 전처리 및 모델 학습

#1. konlpy, keras, tensorflow 라이브러리 설치

```
pip install konlpy
```

```
Collecting konlpy
  Downloading konlpy-0.6.0-py2.py3-none-any.whl.metadata (1.9 kB)
Collecting JPype1>=0.7.0 (from konlpy)
  Downloading jpype1-1.6.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (5.0 kB)
Requirement already satisfied: lxml>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from konlpy) (5.4.0)
Requirement already satisfied: numpy>=1.6 in /usr/local/lib/python3.12/dist-packages (from konlpy) (2.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from JPype1>=0.7.0->konlpy) (25.0)
Downloading konlpy-0.6.0-py2.py3-none-any.whl (19.4 MB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 19.4/19.4 MB 34.3 MB/s eta 0:00:00
Downloading jpype1-1.6.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (495 kB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 495.9/495.9 kB 22.5 MB/s eta 0:00:00
Installing collected packages: JPype1, konlpy
Successfully installed JPype1-1.6.0 konlpy-0.6.0
```

```
pip install keras
```

```
pip install tensorflow
```

#2. 데이터 전처리

```
train_data.drop('id', axis=1, inplace=True)
test_data.drop('id', axis=1, inplace=True)
```

```
print(f"(train) len : {len(train_data)}")
print(f"(test) len : {len(test_data)}")
```

```
# document 열의 중복 제거
```

```
train_data.drop_duplicates(subset=['document'], inplace=True)
test_data.drop_duplicates(subset=['document'], inplace=True)
```

```
print(f"(train) len : {len(train_data)}")
print(f"(test) len : {len(test_data)}")
```

```
train_data['label'].value_counts()
```

```
train_data.isnull().sum()
```

#2. 데이터 전처리

```
# 결측치 제거
train_data.dropna(inplace=True)
test_data.dropna(inplace=True)
```

```
# 한글과 공백 제외 리뷰 제거
train_data['document'] = train_data['document'].str.replace("[^ㄱ-ㅎㅌ-ㅣ가-힣 ]", "", regex=True)
test_data['document'] = test_data['document'].str.replace("[^ㄱ-ㅎㅌ-ㅣ가-힣 ]", "", regex=True)
train_data.head()
```

	document	label
0	아 더빙 진짜 짜증나네요 목소리	0
1	흠포스터보고 초딩영화줄오버연기조차 가볍지 않구나	1
2	너무재밌었다그래서보는것을추천한다	0
3	교도소 이야기구먼 솔직히 재미는 없다평점 조정	0

```
# 빈 데이터 제거
train_data = train_data[train_data['document'].str.strip().astype(bool)]
test_data = test_data[test_data['document'].str.strip().astype(bool)]

print(f"(train) len : {len(train_data)}")
print(f"(test) len : {len(test_data)}")
```

결측치 제거

텍스트 정제

: 한글과 공백을 제외한 모든 문자 제거

빈 데이터 제거

#3. 불용어 제거 및 토큰화

```
# 토큰화
okt = Okt()

stopwords = ['의', '가', '이', '은', '들', '는', '좀', '잘', '강', '과', '도', '를', '으로', '자', '에', '와', '한', '하다'] # 불용어
X_train = []

for sentence in tqdm(train_data['document']):
    tokenized_sentence = okt.morphs(sentence, stem=True) # 토큰화
    stopwords_removed_sentence = [word for word in tokenized_sentence if not word in stopwords] # 불용어 제거
    X_train.append(stopwords_removed_sentence)

X_train[:3]
```

100%|██████████| 145393/145393 [04:53<00:00, 495.49it/s]

```
[['아', '더빙', '진짜', '짜증나다', '목소리'],
 ['흠', '포스터', '보고', '초딩', '영화', '줄', '오버', '연기', '조차', '가볍다', '알다'],
 ['너', '무재', '밧었', '다그', '래서', '보다', '추천', '다']]
```

```
• X_test = []
  for sentence in tqdm(test_data['document']):
      tokenized_sentence = okt.morphs(sentence, stem=True) # 토큰화
      stopwords_removed_sentence = [word for word in tokenized_sentence if not word in stopwords] # 불용어 제거
      X_test.append(stopwords_removed_sentence)
```

okt(한국어 형태소 분석기)로
문장 전체를 형태소 단위로 분리

→ 불용어 제거
eg.) 의, 가, 이, 은, 를, 는 ...

→ 훈련 데이터와 테스트 데이터에
동일한 전처리 과정 적용

#4. 희귀 단어 비율 확인

```
# 희귀 단어 비율 확인
tokenizer = Tokenizer()
tokenizer.fit_on_texts(X_train)

threshold = 3
total_cnt = len(tokenizer.word_index) # 단어 수
rare_cnt = 0 # 등장 빈도가 3보다 작은 단어 수
total_freq = 0 # 훈련 데이터의 전체 단어 빈도수 총 합
rare_freq = 0 # 등장 빈도가 3보다 작은 단어의 등장 빈도의 총 합

for key, value in tokenizer.word_counts.items():
    total_freq = total_freq + value

    if(value < threshold):
        rare_cnt = rare_cnt + 1
        rare_freq = rare_freq + value

print('단어수 :',total_cnt)
print('등장 빈도가 %s번 이하인 희귀 단어 수: %s'%(threshold - 1, rare_cnt))
print("희귀 단어 사용 비율:", (rare_freq / total_freq)*100)
```

단어수 : 43752
등장 빈도가 2번 이하인 희귀 단어 수: 24337
희귀 단어 사용 비율: 1.8715872104872904

희귀어 분석

: 오타, 비속어 변형, 잘못 쓴 단어 등을 제외하여
모델의 학습 효율과 일반화 성능을 향상

#5. 정수 인코딩 및 패딩

```
# 인코딩
tokenizer = Tokenizer(total_cnt - rare_cnt + 1) # 희귀 단어 제거
tokenizer.fit_on_texts(X_train)

X_train = tokenizer.texts_to_sequences(X_train)
X_test = tokenizer.texts_to_sequences(X_test)
y_train = np.array(train_data['label'])
y_test = np.array(test_data['label'])
```

```
# 희귀 단어 제거 후 다시 결측치 제거
drop_train = [i for i, sentence in enumerate(X_train) if len(sentence) < 1]

X_train = [x for i, x in enumerate(X_train) if i not in drop_train]
y_train = [y for i, y in enumerate(y_train) if i not in drop_train]

print(len(X_train))
print(len(y_train))
```

정수 인코딩

: 텍스트 데이터를 숫자 시퀀스로 변환하는 단계

희귀 단어 제거 → 최종 단어 사전 생성
→ 모든 문장을 단어ID의 시퀀스로 변환

#5. 정수 인코딩 및 패딩

```
# 리뷰 길이 분포
print('리뷰의 평균 길이 :', sum(map(len, X_train))/len(X_train))
plt.hist([len(review) for review in X_train], bins=50)
plt.xlabel('length of samples')
plt.ylabel('number of samples')
plt.show()
```

리뷰의 평균 길이 : 10.812485361182679

```
# 길이 이하 리뷰 비율
def below_threshold_len(max_len, nested_list):
    count = 0
    for sentence in nested_list:
        if(len(sentence) <= max_len):
            count = count + 1
    print('전체 샘플 중 길이가 %s 이하인 샘플의 비율: %s'%(max_len, (count / len(nested_list))*100))

below_threshold_len(20, X_train)
below_threshold_len(25, X_train)
below_threshold_len(30, X_train)
```

전체 샘플 중 길이가 20 이하인 샘플의 비율: 88.18836885686336

전체 샘플 중 길이가 25 이하인 샘플의 비율: 91.77884019233684

전체 샘플 중 길이가 30 이하인 샘플의 비율: 94.31944999380003

모든 입력 문장의 길이를 동일하게 맞춰야 하므로,
문장 길이의 분포를 분석한 후,
적절한 최대 길이를 결정

#5. 정수 인코딩 및 패딩

```
# 패딩  
max_len = 30  
X_train = pad_sequences(X_train, maxlen=max_len)  
X_test = pad_sequences(X_test, maxlen=max_len)
```

패딩

: 모든 문장의 길이를 동일하게 만들어서
모델이 처리할 수 있도록 하는 전처리 과정

6. 데이터비율(0.3,0.5,0.7,1.0)에 따른 모델학습/성능비교

```
def build_model(hp):
    model = Sequential()

    embedding_dim = hp.Choice('embedding_dim', values=[50, 100, 200])
    model.add(Embedding(total_cnt - rare_cnt + 1, embedding_dim))
    units = hp.Int('lstm_units', min_value=64, max_value=256, step=64)
    model.add(Bidirectional(LSTM(units)))
    model.add(Dense(1, activation='sigmoid'))

    # 최적화기 선택
    optimizer = hp.Choice('optimizer', ['adam', 'rmsprop'])
    model.compile(
        optimizer=optimizer,
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model

# 데이터 양 조절 실험 + 하이퍼파라미터 튜닝
data_ratios = [0.3, 0.5, 0.7, 1.0]
results = []

for ratio in data_ratios:
    print(f"\n=== 데이터 비율: {int(ratio * 100)}% ===")
```

```
# 데이터 샘플링
sampled_data = train_data.sample(frac=ratio, random_state=42)
X_train_sample = tokenizer.texts_to_sequences(sampled_data['document'])
X_train_pad = pad_sequences(X_train_sample, maxlen=max_len)
y_train_sample = np.array(sampled_data['label'])

# 튜너 설정
tuner = RandomSearch(
    build_model,
    objective='val_accuracy',
    max_trials=3,                    # 실험 횟수 조절 가능
    executions_per_trial=1,
    directory='tuner_results',
    project_name=f'sentiment_ratio_{int(ratio*100)}'
)

es = EarlyStopping(monitor='val_loss', patience=2, verbose=1)
```

정수 인코딩, 패딩, label 추출

비율(0.3,0.5,0.7,1.0)에 맞는 데이터 추출 ⇒ 모델 설계, 실험 데이터 준비, 하이퍼파라미터 튜닝 환경 구성

7. Keras Tuner로 하이퍼파라미터 탐색

```
# 하이퍼파라미터 탐색
tuner.search(
    X_train_pad, y_train_sample,
    epochs=5,
    validation_split=0.2,
    batch_size=64,
    callbacks=[es],
    verbose=1
)

# 최적 모델로 평가
best_model = tuner.get_best_models(num_models=1)[0]
loss, acc = best_model.evaluate(X_test, y_test, verbose=0)
results.append((ratio, acc))
print(f"테스트 정확도 ({int(ratio*100)}% 데이터): {acc:.4f}")
```

최적의 모델 탐색 후, 실제 모델 학습 진행

가장 성능 좋은 모델을 뽑은 후,
테스트 데이터 정확도 측정

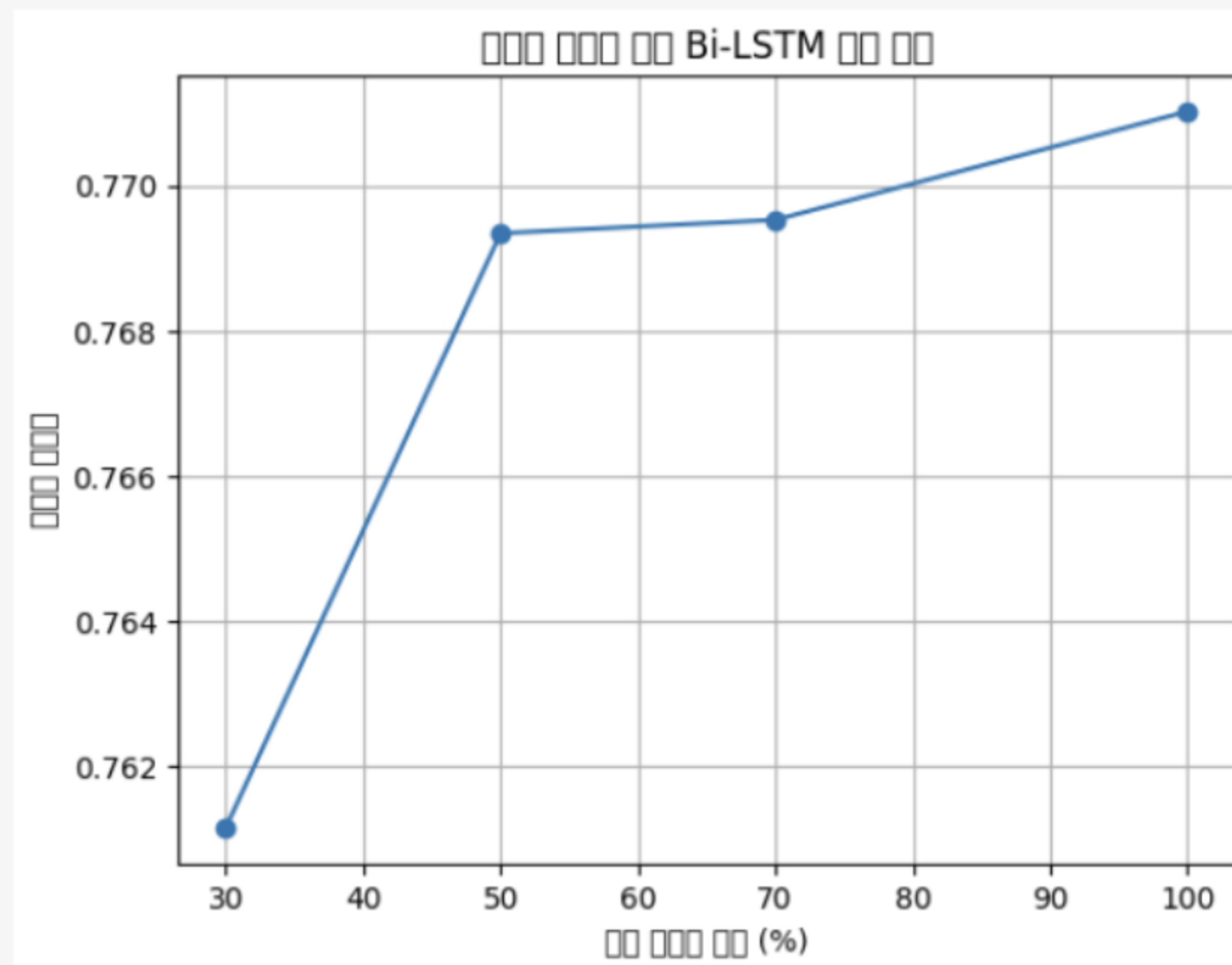
04 예측 테스트를 통한 결과 확인

8. 결과 확인 및 시각화

```
# 결과 시각화
import matplotlib.pyplot as plt

ratios = [int(r[0]*100) for r in results]
accuracies = [r[1] for r in results]

plt.plot(ratios, accuracies, marker='o')
plt.title('데이터 비율에 따른 Bi-LSTM 성능 변화')
plt.xlabel('학습 데이터 비율 (%)')
plt.ylabel('테스트 정확도')
plt.grid(True)
plt.show()
```



데이터 비율에 따른 Bi-LSTM 성능 변화

⇒ 학습 데이터 양이 늘어날 수록 성능이 좋아짐을 확인할 수 있음.

9. 리뷰 길이별 정확도 비교

```
# 리뷰 길이별 정확도 확인
length_bins = [0, 10, 20, 30] # 길이 구간
X_test_lens = np.sum(X_test > 0, axis=1) # 패딩 제외 실제 길이

for i in range(len(length_bins)-1):
    idx = [j for j, l in enumerate(X_test_lens) if length_bins[i] < l <= length_bins[i+1]]
    if len(idx) == 0:
        continue
    acc = np.mean((best_model.predict(X_test[idx])>0.5).flatten() == y_test[idx])
    print(f"{length_bins[i]+1}-{length_bins[i+1]} 길이 리뷰 정확도: {acc:.4f}")
```

9. 리뷰 길이별 정확도 비교

```
Trial 3 Complete [00h 01m 27s]  
val_accuracy: 0.680078387260437
```

```
Best val_accuracy So Far: 0.6810756921768188
```

```
Total elapsed time: 00h 03m 34s
```

```
테스트 정확도 (100% 데이터): 0.7710
```

```
981/981 ————— 4s 4ms/step
```

```
1-10 길이 리뷰 정확도: 0.7727
```

```
363/363 ————— 1s 3ms/step
```

```
11-20 길이 리뷰 정확도: 0.7714
```

```
180/180 ————— 1s 3ms/step
```

```
21-30 길이 리뷰 정확도: 0.7668
```

전체 데이터를 이용하여 학습한 결과,
테스트 데이터 기준으로 77.1%의 정확도를 도출

1-10 길이 리뷰 정확도: 0.7727

11-20 길이 리뷰 정확도: 0.7714

21-30 길이 리뷰 정확도: 0.7668

길이가 짧은 리뷰일 수록
더욱 높은 정확도를 갖추고 있음을 확인할 수 있음.

10. 틀린 리뷰 샘플 확인 (오류 확인)

```
# 틀린 리뷰 샘플 확인 (오류 분석)
preds = best_model.predict(X_test)
wrong_idx = [i for i, (p, y) in enumerate(zip(preds, y_test)) if (p>0.5)!=y]
print("틀린 리뷰 샘플 예시:", test_data['document'].iloc[wrong_idx[:5]])
```

1527/1527 ————— 4s 3ms/step

틀린 리뷰 샘플 예시: 13

15

25 괜히 나올 때 어미 알 건드려서 굶어 부스름 만들었다는 분이 저 아래 보이던데 영화...

26

33

Name: document, dtype: object

$p > 0.5 \rightarrow$ 모델의 예측값 (True/False)

모델이 잘못 예측한 리뷰를 추출하여,
잘못 분류된 문장의 특성을 분석

$\rightarrow$

$y \rightarrow$ 실제 정답 (0 또는 1)

$\Rightarrow$

반어법, 애매한 표현, 문맥 의존성이 강한 문장에서
모델이 어려움을 겪는 것을 확인할 수 있음.

둘이 다르면 $\rightarrow$ 틀린 예측

11. 결론: 감성 예측 함수 도출 및 예측 테스트

```
# 감성 예측 함수
def sentiment_predict(new_sentence):
    new_sentence = re.sub(r'^ㄱ-ㅎㅏ-ㅣ가-힣 ]', '', new_sentence)
    new_sentence = okt.morphs(new_sentence, stem=True) # 토큰화
    new_sentence = [word for word in new_sentence if not word in stopwords] # 불용어 제거
    encoded = tokenizer.texts_to_sequences([new_sentence]) # 인코딩
    pad_new = pad_sequences(encoded, maxlen=max_len) # 패딩
    score = float(best_model.predict(pad_new)) # 예측

    if score > 0.5:
        print("{:.2f}% 확률로 긍정 리뷰입니다.\n".format(score * 100))
    else:
        print("{:.2f}% 확률로 부정 리뷰입니다.\n".format((1 - score) * 100))
```

예측 테스트 (최종 모델 기준)

```
sentences = [
    "이 영화 개꿀잼 ㅋㅋㅋ",
    "이 영화 핵노잼 ㅠㅠ",
    "연기가 정말 훌륭했어요",
    "스토리가 지루해서 잠옴",
    "개노잼이네 ㅋㅋ"
]
```

```
for s in sentences:
    sentiment_predict(s)
```

1/1 ————— 0s 30ms/step
87.82% 확률로 긍정 리뷰입니다.

1/1 ————— 0s 30ms/step
97.33% 확률로 부정 리뷰입니다.

1/1 ————— 0s 35ms/step
97.67% 확률로 긍정 리뷰입니다.

최적의 Bi-LSTM 모델을 이용하여 새로운 리뷰 문장에 대한 감성 예측을 수행한 결과,
대부분의 문장을 높은 확률로 긍정/부정으로 정확히 분류하는 것을 확인할 수 있었음.

05 Bi-LSTM + Attention 모델

Attention 이란?

입력 시퀀스가 길어지면 출력 시퀀스의 정확도가 떨어지는 것을 보정해주기 위한 등장한 기법

RNN / LSTM

긴 문장을 하나의 벡터에 다 압축해서 기억함.
앞부분은 잊어버리고
중요한 단어가 뒤에 있으면 놓치기도 함.



Attention

문장 전체를 볼 뿐만 아니라 각각을 똑같이 보는 게 아니라 중요해 보이는 단어에 더 큰 집중하는 방법

Bi-LSTM + Attention

Bi-LSTM은 문장을 양쪽에서 모두 읽어주고,

Attention은 그 문장 속에서 중요한 부분에 더 큰 가중치를 준다. (=더 집중하게 해준다.)

eg.) 연기는 좋았는데, 스토리는 도무지 이해가 되지 않았다.

Bi-LSTM 역할

앞: 연기는 좋았는데
뒤: 스토리는 도무지 이해가 되지 않았다.

Attention 역할

“이해가 되지 않았다”에 더 높은 가중치

Bi-LSTM+Attention ⇒ **감정 표현을 더 정확하게 잡아낼 수 있음.**

#2. Attention 레이어 정의

```
# Attention Layer 정의
from tensorflow.keras.layers import Layer
import tensorflow.keras.backend as K

class Attention(Layer):
    def __init__(self, **kwargs):
        super(Attention, self).__init__(**kwargs)

    def build(self, input_shape):
        self.W = self.add_weight(name='att_weight',
                                   shape=(input_shape[-1], 1),
                                   initializer='random_normal',
                                   trainable=True)
        self.b = self.add_weight(name='att_bias',
                                   shape=(input_shape[1], 1),
                                   initializer='zeros',
                                   trainable=True)
        super(Attention, self).build(input_shape)

    def call(self, x):
        e = K.tanh(K.dot(x, self.W) + self.b)
        a = K.softmax(e, axis=1)
        output = x * a
        return K.sum(output, axis=1)
```

Bi-LSTM이 시간축마다
출력한 hidden state들 중에서
어떤 시점에 더 집중할지
가중치를 계산해 줌.

시간축 = 단어 순서, 시점 = 단어

⇒ Bi-LSTM이 만들어낸 단어별 표현들 중에서
중요한 단어에 더 높은 가중치를 줘서,
문장을 하나의 벡터로 요약해줌

#3. Attention이 포함된 Bi-LSTM 모델 정의

```
# Attention 결합 모델
def build_model_with_attention(hp):
    model = Sequential()
    embedding_dim = hp.Choice('embedding_dim', values=[50, 100, 200])
    model.add(Embedding(total_cnt - rare_cnt + 1, embedding_dim))
    units = hp.Int('lstm_units', min_value=64, max_value=256, step=64)
    model.add(Bidirectional(LSTM(units, return_sequences=True)))
    model.add(Attention())
    model.add(Dense(1, activation='sigmoid'))

    optimizer_name = hp.Choice('optimizer', ['adam', 'rmsprop'])
    optimizer = Adam() if optimizer_name == 'adam' else RMSprop()

    model.compile(optimizer=optimizer,
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
    return model
```

Bi- LSTM으로 문맥을 읽고,
Attention으로 중요한 단어에 집중한 다음,
최종적으로 리뷰의 감정을 0~1 확률로
예측하는 모델 구조 정의

#4. 튜닝 및 평가 함수

```
# 튜닝 및 평가 함수
def train_and_evaluate(model_builder, project_name):
    tuner = RandomSearch(
        model_builder,
        objective='val_accuracy',
        max_trials=3,
        executions_per_trial=1,
        directory='tuner_results',
        project_name=project_name,
        overwrite=True
    )
    es = EarlyStopping(monitor='val_loss', patience=2, verbose=1)

    tuner.search(
        X_train, y_train,
        epochs=5,
        validation_split=0.2,
        batch_size=64,
        callbacks=[es],
        verbose=1
    )

    best_model = tuner.get_best_models(num_models=1)[0]
    loss, acc = best_model.evaluate(X_test, y_test, verbose=0)
    return best_model, acc
```

모델의 설정값을 변경해보면서
검증 정확도가 가장 높은 모델을 선택한 후,
테스트 성능까지 확인하는 튜닝+평가 함수를 정의

#5. -데이터비율(0.3,0.5,0.7,1.0)에 따른 모델학습/성능비교

```
data_ratios = [0.3, 0.5, 0.7, 1.0]
results = []

for ratio in data_ratios:
    print(f"\n=== 데이터 비율: {int(ratio * 100)}% ===")

    # 데이터 샘플링
    indices = np.random.choice(len(X_train), size=int(len(X_train)*ratio), replace=False)
    X_train_sample = X_train[indices]
    y_train_sample = y_train[indices]

    # 기본 Bi-LSTM 모델
    print("=== Base Bi-LSTM 모델 학습 ===")
    base_model, base_acc = train_and_evaluate(build_model, 'base_bilstm')

    # Attention 추가 Bi-LSTM 모델
    print("\n=== Bi-LSTM + Attention 모델 학습 ===")
    att_model, att_acc = train_and_evaluate(build_model_with_attention, 'bilstm_attention')

    print(f"기존 Bi-LSTM 정확도: {base_acc:.4f}")
    print(f"Bi-LSTM + Attention 정확도: {att_acc:.4f}")

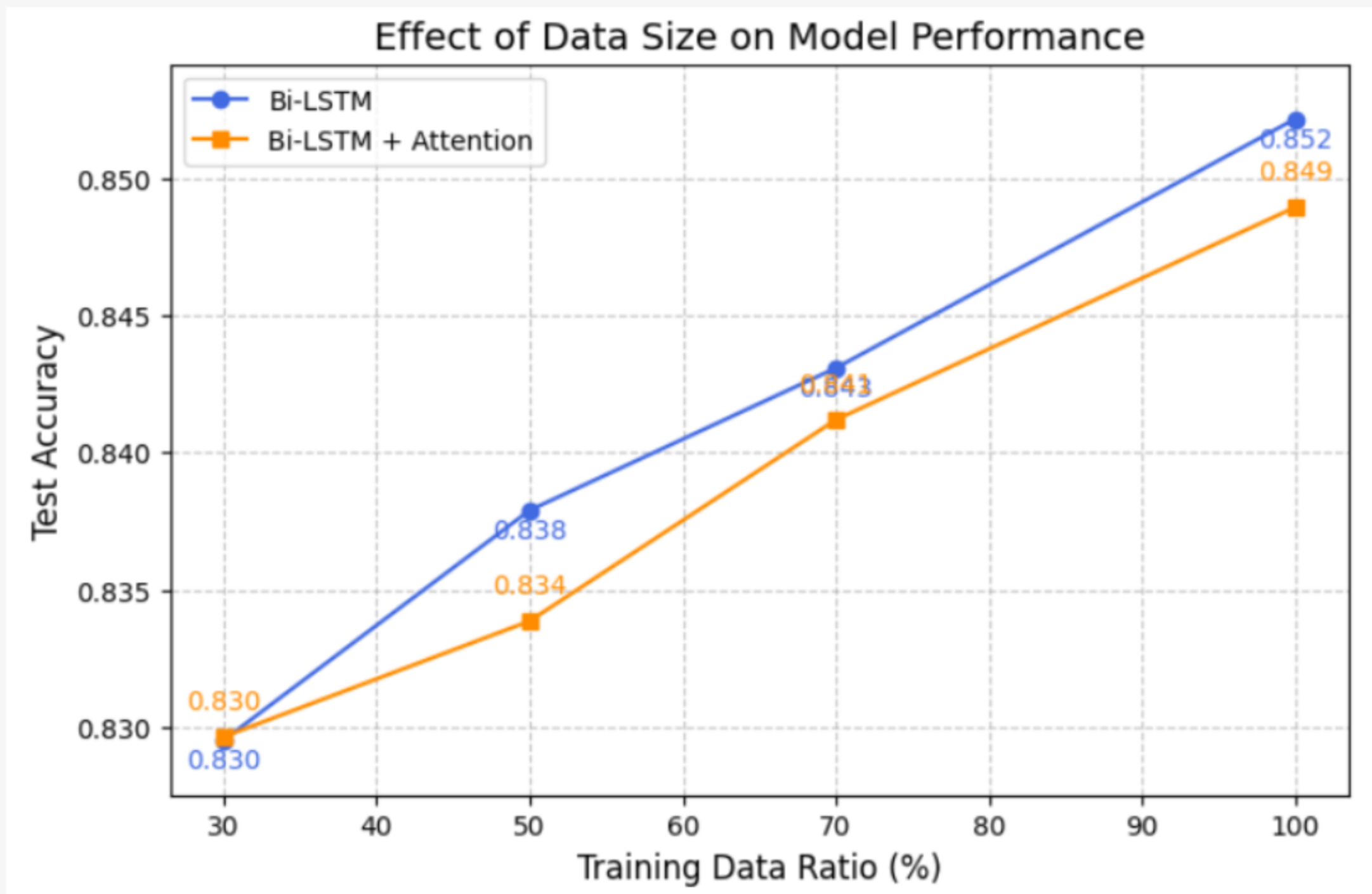
    results.append((ratio, base_acc, att_acc))
```

Bi-LSTM 모델에서와 같이

Attention을 결합한 모델도

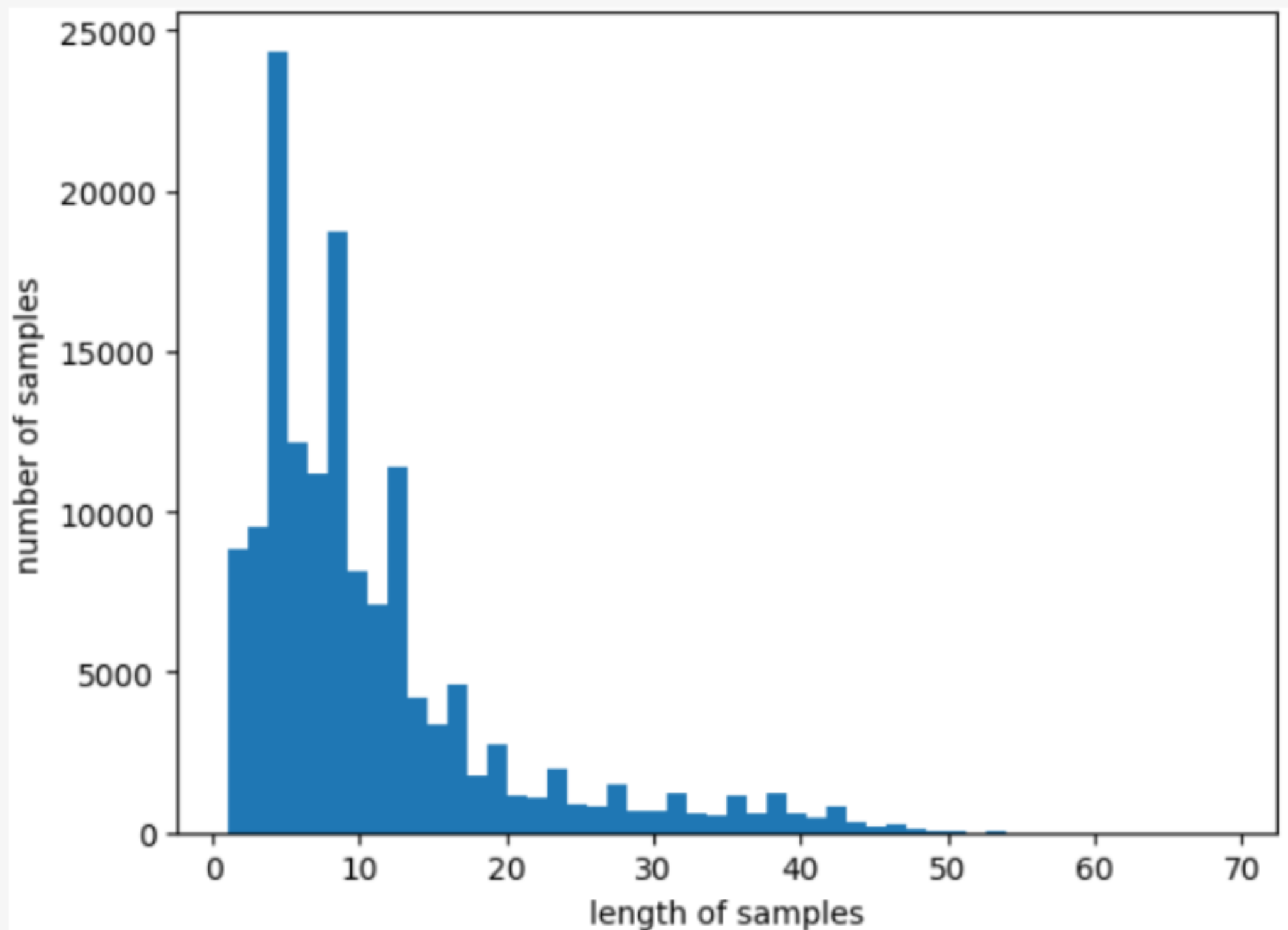
데이터비율에 따른 모델학습/성능비교 단계 수행

#6. 기존 모델과 Attention 결합 모델의 정확도 비교



오히려 Bi-LSTM이 아주 미세하지만
높은 성능을 보임
⇒ 데이터의 특성

#6. 기존 모델과 Attention 결합 모델의 정확도 비교



데이터 셋 내 문장의 평균 길이가 약 10.8개의
단어로 이루어져 있음
⇒ 병목 현상 존재 X

질의응답

감사합니다.